

Modern Yazılım Mimarilerinde Kimlik Doğrulama, Kullanıcı Profili Yönetimi ve Çevrimdışı Senkronizasyon Stratejileri

Kimlik Doğrulama Paradigmalarının Evrimi ve Mimarinin Temelleri

Gelişmiş yazılım projelerinde, özellikle halihazırda faaliyette olan veya belirli bir olgunluğa erişmiş sistemlere yeni bir kimlik doğrulama (authentication) ve yetkilendirme (authorization) fazının entegre edilmesi, yalnızca teknik bir kodlama sürecinden ibaret değildir. Bu süreç; kullanıcı deneyimini (UX), veri güvenliğini, yasal uyumluluk çerçevelerini ve sistemin genel performansını derinden etkileyen stratejik bir dönüşüm hamlesidir. Günümüz endüstri standartlarında, yazılım mimarileri tasarlanırken kullanıcıların dijital platformlara erişim sağlarken karşılaştıkları engellerin, bir diğer deyişle sürtünme katsayısının (friction) en aza indirilmesi birincil hedef olarak belirlenmektedir. Geleneksel yazılım geliştirme pratiklerinde sıkça karşılaşılan ve kullanıcıların sisteme giriş yapabilmek için uzun, karmaşık formları doldurmak zorunda bırakıldığı monolitik kayıt süreçleri, yerini artık Sosyal Oturum Açma (Social Login / SSO) mekanizmalarına ve asgari bilgi talebine dayalı kademeli profillemeye (progressive profiling) stratejilerine bırakmıştır.

Piyasada genel kullanıcı eğilimleri ve büyük teknoloji şirketlerinin (Big Tech) uyguladığı pratikler incelendiğinde, dijital ürünlerle etkileşime giren bireylerin sabır eşiklerinin giderek düştüğü görülmektedir. Özellikle mobil uygulamalarda veya modern web platformlarında, bir kullanıcının sisteme kayıt olma aşamasında yaşayacağı her bir saniyelik gecikme veya doldurulması talep edilen her bir fazladan veri alanı, doğrudan müşteri kaybı (churn rate) ve dönüşüm oranlarında (conversion rate) ciddi düşüşlerle sonuçlanmaktadır. Bu bağlamda, Google ile giriş (Google OAuth 2.0) ve e-posta tabanlı kayıt yöntemlerinin bir arada sunulması, platformun kapsayıcılığını artırmak adına vazgeçilmez bir endüstri standardı haline gelmiştir. Ancak, bu entegrasyonun arka planında çalışacak olan kullanıcı profili oluşturma ve veri tutma mantığı; örneğin hangi verilerin tam olarak hangi aşamada talep edileceği, Türkiye Cumhuriyeti Kimlik Numarası (TCKN), araç plakası ve şasi numarası (VIN) gibi hassas verilerin nasıl saklanacağı ve bu verilerin internet bağlantısının olmadığı çevrimdışı (offline-first) senaryolarda nasıl senkronize edileceği gibi hususlar, derinlemesine bir mühendislik ve veri yönetişimi yaklaşımını zorunlu kılmaktadır.

Özellikle araç verileri ve yasal kimlik belirteçlerinin toplanması, Kişisel Verilerin Korunması Kanunu (KVKK) ve Avrupa Birliği Genel Veri Koruma Tüzüğü (GDPR) kapsamında son derece sıkı hukuki yükümlülükler doğurmaktadır. Bu verilerin gereksiz yere toplanması, sistemin yasal bir risk havuzuna dönüşmesine sebebiyet verebilir. Diğer taraftan sistemin arka planında çalışacak olan Supabase ve PowerSync gibi modern mimariler, bu verilerin cihazlar arasında güvenli, tutarlı ve şifrelenmiş bir biçimde taşınmasını sağlayan oldukça karmaşık protokoller içermektedir. İşbu kapsamlı araştırma raporu, mevcut bir faza entegre edilecek olan kimlik doğrulama sisteminin endüstri standartlarına en uygun, kullanıcı tarafında en mantıklı ve hukuki açıdan en sorunsuz şekilde nasıl kurgulanması gerektiğini analiz etmektedir. İnceleme; kullanıcı

deneyimi psikolojisi, veri minimizasyonu ilkeleri, ideal ilişkisel veritabanı şeması tasarımları ve Supabase/PowerSync tabanlı çevrimdışı öncelikli mimarilerin teknik derinlikleri üzerinden aşama aşama gerçekleştirilecektir.

Kullanıcı Deneyimi Psikolojisi ve Modern Oturum Açma Yöntemleri

Bir kullanıcının bir yazılım sistemiyle etkileşimindeki ilk ve tartışmasız en kritik temas noktası, kimlik doğrulama arayüzleridir. Genel endüstri uygulamalarına bakıldığında, başarısız platformların kayıt ekranlarını bir "veri madenciliği" aracı olarak gördüğü, başarılı platformların ise kayıt ekranını yalnızca "kullanıcıya bir anahtar teslim etme" süreci olarak tasarladığı görülmektedir. Sistem tasarımında "genel kullanıcılar nasıl yapıyor, biz nasıl yapmalıyız" sorusunun en yalın cevabı, giriş bariyerini sıfıra yaklaştıran güvene dayalı bir model inşa etmektir.

Google ile Giriş (OAuth 2.0 ve OpenID Connect) Dinamikleri

Sisteme entegre edilmesi kesin olarak hedeflenen "Google ile Giriş" seçeneği, kullanıcıların büyük bir çoğunluğunun yaşadığı şifre yorgunluğunu (password fatigue) ortadan kaldıran en etkili mekanizmadır. Google OAuth 2.0 altyapısı, kullanıcının cihazında halihazırda doğrulanmış olan oturum üzerinden çalışarak, kimlik doğrulama yükünü uygulamanın kendi sunucularından alıp Google'ın yüksek güvenli altyapısına devreder. Bu işlem sırasında uygulama, Google sunucularına bir yetkilendirme talebi gönderir ve kullanıcı bu talebe onay verdiğinde, Google tarafından sisteminize güvenli bir JSON Web Token (JWT) iletilir. Bu yöntemle kullanıcının doğrulanmış e-posta adresi, adı, soyadı ve varsa profil fotoğrafı milisaniyeler içinde çekilerek veritabanında anında bir kayıt oluşturulur. Sistem bu sayede, kullanıcının sahte bir e-posta adresi girmesi riskini ortadan kaldırır ve e-posta doğrulama (verification) adımını tamamen atlayarak kullanıcıyı doğrudan uygulamanın ana ekranına (dashboard) ulaştırır. Modern bir Supabase kurulumunda bu süreç, Supabase Auth modülü üzerinden dış kimlik sağlayıcıların (External Auth Providers) aktif edilmesiyle tek merkezden yönetilebilir hale gelmektedir.

E-Posta Tabanlı Kimlik Doğrulama Stratejileri

Google ekosistemi dışında kalmayı tercih eden, mahremiyet odaklı hareket eden veya iş süreçleri gereği kurumsal e-posta adreslerini kullanmak isteyen kullanıcılar için e-posta tabanlı bir giriş sistemi sunulması zorunludur. Ancak bu noktada genel kullanıcıların sıkça yaptığı hatalara düşülmemelidir. Geleneksel sistemler, kullanıcıdan e-posta adresi ile birlikte en az sekiz karakter uzunluğunda, içinde büyük harf, küçük harf, rakam ve özel karakter barındıran kompleks şifreler yaratmasını talep eder ve ardından e-postasına gidip bir aktivasyon bağlantısına tıklamasını zorunlu kılar. Bu durum, mobil cihazlarda uygulama pencereleri arasında geçiş yapmayı gerektirdiği için dönüşüm hunisinde (conversion funnel) ciddi kopmalara yol açar.

Endüstri standardında en mantıklı ve çağdaş e-posta giriş yöntemi, "Sihirli Bağlantı" (Magic Link) veya "Tek Kullanımlık Şifre" (OTP - One Time Password) mimarileridir. Bu kurguda kullanıcı sadece e-posta adresini girer; uygulama kullanıcının e-posta adresine güvenli bir link veya altı haneli bir kod gönderir. Kullanıcı bu koda tıkladığında veya kodu ekrana girdiğinde Supabase Auth sistemi arka planda otomatik olarak bir oturum token'i (session token) üretir ve

kullanıcıyı içeri alır. Böylece şifre unutma, şifre sıfırlama, karmaşık şifre kurallarına uyum sağlama gibi kullanıcıyı yoran operasyonel yükler tamamen sistemden silinir. Uygulama geliştiricisi olarak kurgulamanız gereken yapı, kullanıcının bilişsel yükünü (cognitive load) sıfırlayan bu tür modern ve şifresiz (passwordless) deneyimleri merkeze almalıdır.

Kademeli Profilleme (Progressive Profiling) ve Optimal Veri Toplama

Oturum açma aşamasında kullanıcıdan hangi bilgilerin istenmesi gerektiği konusu, sistemin gelecekteki ölçeklenebilirliği, kullanıcı güveni ve yasal regülasyonlar bağlamında projenin en hayati karar noktasıdır. Talep edilen e-posta, telefon, TCKN, araç plakası ve şasi numarası gibi bilgilerin tamamının ilk kayıt ekranında bir form yığını halinde kullanıcıya sunulması, modern yazılım mimarisi prensiplerine tamamen aykırıdır. Bu yaklaşım, kullanıcının henüz uygulamanın kendisine sunacağı değeri (value proposition) tam olarak idrak etmeden sisteme karşı savunmaya geçmesine neden olur.

En mantıklı, en uygun ve kullanıcı profilini en sağlıklı şekilde doldurmaya yarayan yöntem **Kademeli Profilleme (Progressive Profiling)** stratejisidir. Bu strateji, kullanıcının psikolojik bariyerlerini aşmak amacıyla verileri tek bir seferde değil, zamana yayarak ve sadece işlemin doğasının gerektirdiği tam o kritik anlarda (just-in-time) talep etmeyi ifade eder.

Sıfır Güven (Zero-Trust) ve Asgari Giriş Fazı

Bir kullanıcının sistemin veritabanında bir "kimlik" (identity) ve bir eşsiz tanımlayıcı (UUID) edinebilmesi için kayıt anında (onboarding) talep edilecek yegane bilgi dizisi e-posta adresi (veya Google hesabı üzerinden alınan ad/soyad bilgisi) olmalıdır. Bu aşamada telefon numarası, kimlik numarası veya araca dair herhangi bir bilgi kesinlikle sorulmamalıdır. Birinci fazın tek amacı, kullanıcıyı sistemin içine çekmek ve uygulamanın arayüzü ile etkileşime girmesini sağlamaktır. Veritabanı şemasında bu bilgiler Supabase tarafından güvenli bir şekilde auth.users isimli kapalı bir şemada şifreli olarak tutulur ve uygulamanın genel iş mantığının koştugu public şemasındaki tablolardan tamamen izole edilir.

Bağlamsal Veri Toplama (Context-Aware Data Gathering) Fazları

Kullanıcı uygulamanın ana ekranına ulaştıktan sonra, yapmak istediği işlemlere bağlı olarak diğer kişisel veriler kendisinden gerekçelendirilerek talep edilmelidir. Kademeli profilleme mimarisi şu adımlarla kurgulanmalıdır:

1. **Telefon Numarası Entegrasyonu:** Kullanıcı, profilinin güvenliğini artırmak amacıyla İki Faktörlü Kimlik Doğrulama (2FA) özelliğini aktif etmek istediğinde veya uygulamanın temel bir işlevi gereği anlık SMS bildirimleri (örneğin otopark süresinin dolması, aracın servisten çıkması) almayı talep ettiğinde telefon numarası istenir. Bu bilgi, kullanıcı kayıt olurken "İleride lazım olur" mantığıyla değil, doğrudan bir fayda karşılığında elde edilir.
2. **Araç Plakası ve Şasi Numarası (VIN) Kaydı:** Uygulamanın en temel modüllerinden biri olan "Araçlarım" veya "Garaj" bölümüne girildiğinde, sistem kullanıcıdan "Yeni Bir Araç Ekle" aksiyonunu almasını bekler. İşte tam bu bağlamda araç plakası, marka, model ve araca ait benzersiz kimlik numarası olan Şasi Numarası (VIN - Vehicle Identification Number) talep edilmelidir. Bu veriler, kullanıcının ana profili içinde değil, ilişkisel veritabanı mantığına uygun olarak vehicles adında ayrı bir tabloda tutulur. Bu tablodaki user_id

sütunu, kullanıcının kimlik numarasına (foreign key) bağlanarak "bire-çok" (one-to-many) bir ilişki modeli kurulur.

3. **TCKN (Türkiye Cumhuriyeti Kimlik Numarası):** Bireyin en hassas kimlik belirteci olan TCKN, uygulamanın yalnızca yasal zorunluluklar doğurduğu noktalarda devreye girmelidir. Kullanıcı uygulama üzerinden resmi bir finansal işlem yapacağı, elektronik ödeme gerçekleştireceği veya sistemin kendisine e-fatura kesmesi gerektiği anda TCKN istenmelidir. Uygulama içinde sadece bilgilendirme alan bir kullanıcıdan asla TCKN istenmez.
4. **Aklına Gelen Diğerleri (İkincil ve Örtük Veriler):** İlk bakışta göze çarpmayan ancak modern profilemede hayati olan diğer veri kategorileri de sistemin arka planında toplanır veya kademeli olarak istenir. Bunlar; pazarlama onayları (açık rıza metinleri), tercih edilen dil, uygulamanın kurulu olduğu cihazın benzersiz tanımlayıcısı (Device ID), anlık bildirim (Push Notification) token'ları, konum (GPS) bilgileri ve kullanıcının uygulama içindeki davranışsal loglarıdır. Cihaz ID'si ve işletim sistemi versiyonu gibi örtük (implicit) veriler oturum açıldığı an sistem tarafından arka planda otomatik olarak toplanırken; konum veya pazarlama izni gibi açık rıza gerektiren (explicit) veriler ise sadece uygulamanın ilgili modülüne girildiğinde bir sistem pop-up'ı (izin penceresi) vasıtasıyla talep edilir.

Aşağıdaki tablo, söz konusu kademeli veri toplama stratejisinin ideal veritabanı şemasına ve tetikleyici işlemlere göre nasıl yapılandırılması gerektiğini göstermektedir:

Toplanan Veri Seti	Verinin Talep Edildiği An / Tetikleyici Aksiyon	Kullanıcı Deneyimi Gerekçesi (UX Justification)	Veritabanı Hedef Tablosu ve İlişkisi
E-Posta / Ad Soyad	1. Kayıt Ekranı (İlk Açılış)	Hesap oluşturma ve sisteme giriş yapabilme.	auth.users ve public.profiles
Cihaz ID / OS Logları	1. Kayıt Ekranı (Arka Plan)	Güvenlik, hata ayıklama, oturum yönetimi.	public.session_logs (System Generated)
Telefon Numarası	2. Bildirim Ayarları veya 2FA	Kritik bildirimlerin iletilmesi ve hesap güvenliği.	public.profiles (Update işlemi)
Araç Plakası / Marka	3. Yeni Araç Ekleme Modülü	Otopark/servis hizmetlerinden yararlanma.	public.vehicles (Foreign Key: user_id)
Şasi Numarası (VIN)	3. Yeni Araç Ekleme Modülü	Araçın teknik tespiti ve yedek parça uyumluluğu.	public.vehicles (Foreign Key: user_id)
TCKN / Vergi No	4. Ödeme / Fatura Adımı	Resmi e-fatura düzenlenmesi ve yasal bildirim.	public.billing_details (Foreign Key: user_id)
Konum (GPS) İzni	5. Harita / Servis Arama Modülü	En yakın istasyonun veya servisin bulunması.	Anlık işlenir, cihaz tabanlı tutulur.

Yasal Çerçeveler: KVKK ve GDPR Kapsamında Veri Minimizasyonu

Yazılım geliştirme süreçlerinde veritabanı mimarisini tasarlariken, teknik gereksinimler kadar ulusal ve uluslararası hukuk kurallarının getirdiği yükümlülükler de göz önünde bulundurulmalıdır. Özellikle Türkiye sınırları içerisinde Kişisel Verilerin Korunması Kanunu (KVKK) ve global ölçekte operasyon hedefleniyorsa Avrupa Birliği Genel Veri Koruma Tüzüğü (GDPR) kuralları, toplanacak verilerin sınırlarını kesin çizgilerle belirlemektedir. Tasarım aşamasında aklımıza gelen her türlü veriyi (TCKN, şasi numarası, iletişim geçmişi vb.) bir form aracılığıyla ilk andan itibaren toplama fikri, "Veri Minimizasyonu" ilkesine açıkça aykırıdır ve ağır idari para cezaları ile sonuçlanabilir.

Veri Minimizasyonu ve Ölçülülük İlkesi

KVKK'nın 4. maddesinde ve GDPR'ın 5. maddesinde temel bir prensip olarak "Kişisel verilerin işlendikleri amaçla bağlantılı, sınırlı ve ölçülü olması" kuralı yer almaktadır. Bu kuralın teknik mimarideki yansıması, bir sürecin işletilmesi için mutlak surette gerekli olmayan hiçbir bilginin, ileride bir gün faydalı olabileceği ihtimaline binaen veri tabanına kaydedilemeyeceği gerçeğidir. Hukuk literatüründe ve Kişisel Verileri Koruma Kurulu (Kurul) kararlarında açıkça ifade edildiği üzere, örneğin Covid-19 pandemisi gibi olağanüstü durumlarda dahi, kamu sağlığının korunması gerekçesiyle uygulanan HES kodu sorgulamalarında Kurul, kodların sadece anlık olarak denetlenmesini tavsiye etmiş ve bu sonuçların kaydedilerek gereğinden fazla sağlık verisi işlenmesinin yasal ilkelere uygun olmayacağını hükme bağlamıştır. Bu örnek, veri sorumlularının (uygulama sahiplerinin) yalnızca sürecin gerektirdiği minimum veri seti ile yetinmesi gerektiğini gösteren en güçlü referanslardan biridir. Dolayısıyla, kullanıcının yalnızca profil oluşturduğu bir aşamada TCKN ve şasi numarası istemek veri minimizasyonu kuralının doğrudan ihlali anlamına gelir.

Araç Şasi Numarası (VIN) ve Plaka Bilgisinin Kişisel Veri Olarak Tasnifi

Araç plakası ve araç şasi numarası (Vehicle Identification Number - VIN) ilk etapta bir taşıta, yani bir eşyaya ait cansız veriler gibi algılanabilir. Ancak hem Avrupa Birliği Adalet Divanı'nın (ECJ) hem de İngiltere Veri Koruma Otoritesi'nin (ICO) güncel ve bağlayıcı içtihatları incelendiğinde, bu verilerin tartışmasız bir biçimde "Kişisel Veri" (Personal Data / PII) niteliği taşıdığı görülmektedir.

İngiltere ICO kılavuzları, bir gerçek kişiyi doğrudan tanımlamasa bile, dolaylı yoldan (indirectly) kimliğinin tespit edilmesine olanak sağlayan, o kişiyi bir grup içerisinde çekip çıkaran (singled out) verileri kişisel veri sayar ve araç kayıt numarasını (plaka) ile VIN bilgisini bu listenin en başına yerleştirir. Daha derin bir hukuki zemin olarak, Avrupa Adalet Divanı'nın yakın zamanda tır üreticisi Scania ile bir Alman ticaret birliği arasında görülen davada (Scania Davası) verdiği karar son derece kritiktir. Mahkeme, bir şasi numarasının kendi başına (izole bir metal parçası üzerindeyken) kişisel veri olmayabileceğini belirtmiş; ancak eğer veri sorumlusu veya herhangi bir üçüncü kişi, makul araçlar (reasonable means) kullanarak bu şasi numarasını bir gerçek kişi ile eşleştirebiliyorsa, o şasi numarasının artık yasal koruma kalkını altındaki bir kişisel veriye dönüştüğüne hükmetmiştir. Sizin kurgulayacağınız modern sistemde, kullanıcı aracı kendi dijital profiline eklediğinde şasi numarası ve plaka otomatik olarak o bireyin e-posta adresi ve ismiyle eşleşecektir. Bu eşleşme sağlandığı andan itibaren veritabanındaki her bir VIN kaydı, bireyin lokasyonunu, ekonomik durumunu veya yaşam tarzını analiz etmeye yarayabilecek bir kişisel veri statüsü kazanır.

Bu durumun en büyük mimari yansıması, kullanıcılardan gelecek "Unutulma Hakkı" (Right to be Forgotten) taleplerinde ortaya çıkar. Kullanıcı hesabının silinmesini talep ettiğinde, GDPR Madde 17 uyarınca sadece users tablosundaki isminin değil, vehicles tablosundaki o kişiye bağlı tüm şasi numaraları ve plakaların da kalıcı olarak silinmesi veya tamamen anonimleştirilmesi gerekmektedir. Bu hukuki gereklilik, veritabanı tasarlanırken silme operasyonlarının basamaklı (cascade delete) şekilde kurgulanmasını zorunlu kılar.

Türkiye Cumhuriyeti Kimlik Numarasının (TCKN) Şifrelenmesi ve İşlenme Şartları

Türkiye Cumhuriyeti Kimlik Numarası (TCKN), veri setleri içerisinde bireyi tekil olarak tespit etmeye yarayan, değiştirilmesi neredeyse imkansız olan yüksek riskli bir veridir. Kurul kararları ışığında TCKN'nin işlenebilmesi için açık rızadan ziyade, kanunlarda açıkça öngörülme şartının aranması çok daha güvenlidir. Elektronik Ticaretin Düzenlenmesi Hakkında Kanun kapsamında mesafeli sözleşmelerin kurulması, Suç Gelirlerinin Aklanmasının Önlenmesi Hakkında Kanun (MASAK) mevzuatı gereği elektronik ödeme işlemleri yapılması veya Kimlik Bildirme Kanunu uyarınca özel tesislerde kayıt tutulması gibi durumlarda TCKN işlenmesi hukuka uygundur. Ancak sadece bir sadakat programına katılmak veya araç bilgilerini takip etmek için TCKN istenmesi Kurul tarafından cezalandırma sebebidir.

Eğer kurgulanan faza bir ödeme geçidi (payment gateway) eklenecekse ve TCKN alınması mecburi ise, bu veri veritabanında kesinlikle düz metin (plaintext) formatında tutulmamalıdır. Veritabanı seviyesinde durağan veri şifrelemesi (encryption at rest) uygulanmalı ve AES-256 gibi güçlü simetrik şifreleme algoritmaları kullanılarak veri tabanı yöneticilerinin dahi sisteme sızması durumunda kimlik numaralarını okuyabilmesi engellenmelidir.

Çevrimdışı Öncelikli (Offline-First) Mimari ve Veritabanı Senkronizasyonu

Modern mobil ve web uygulamalarında karşılaşılan en büyük problemlerden biri, cihazın internet bağlantısının zayıfladığı veya tamamen koptuğu senaryolarda (örneğin yeraltı otoparkları, asansörler veya kırsal bölgeler) uygulamanın yanıt veremez hale gelmesidir. Geleneksel çevrimiçi öncelikli (online-first) mimariler, her bir okuma veya yazma işlemi için doğrudan sunucuya bir ağ isteği gönderir ve yanıt dönene kadar arayüzü kilitletler. Geliştirilecek faza entegre edilecek oturum açma ve profil yönetim sisteminin bu darboğaza düşmemesi için, sektördeki en yenilikçi çözümlerden biri olan **Supabase ve PowerSync** tabanlı çevrimdışı öncelikli (offline-first) senkronizasyon mimarisinin benimsenmesi teknik açıdan en mantıklı karardır.

PowerSync ve Supabase Entegrasyonunun Arka Planı

PowerSync, uygulamanın kurulu olduğu istemci cihazın (mobil telefon, tablet veya tarayıcı) yerel belleğinde gömülü bir SQLite veritabanı örneği oluşturarak çalışır. Kullanıcı uygulamada bir değişiklik yaptığında (örneğin araç plakasını güncellediğinde veya iletişim numarasını değiştirdiğinde), bu veri isteği internet üzerinden backend sunucusuna gitmek yerine doğrudan milisaniyeler içerisinde cihazdaki bu yerel SQLite veritabanına yazılır. Uygulamanın arayüzü bu değişikliği anında algılayıp ekrana yansıtır ve kullanıcı işlemine kesintisiz devam eder. İnternet bağlantısı mevcut olduğunda ise, cihazdaki yerel veritabanı ile bulut ortamındaki merkezi

Supabase (PostgreSQL) veritabanı, PowerSync servisi aracılığıyla arka planda asenkron olarak çift yönlü senkronize edilir.

Bu mimarinin ana motoru, PostgreSQL veritabanı sisteminin sunduğu "Mantıksal Replikasyon" (Logical Replication) özelliğidir. PowerSync, Postgres'in Write-Ahead Log (WAL) adı verilen ve veritabanına yapılan tüm işlemleri kronolojik olarak kaydeden günlük dosyalarını sürekli okuyarak buluttaki değişiklikleri anında algılar. Bu yapıyı kurabilmek için Supabase sunucusunda wal_level = logical ayarının aktifleştirilmesi ve powersync adında özel bir publication (yayın akışı) oluşturularak senkronize edilecek tabloların deklare edilmesi gerekmektedir. Güvenlik açısından ise PowerSync servisinin veritabanına erişebilmesi için yalnızca replikasyon haklarına ve sadece okuma (SELECT) yetkisine sahip sınırlandırılmış bir powersync_role kullanıcısı yaratılmalıdır. Bu yapı sayesinde, sistem milyonlarca eşzamanlı aktif kullanıcısı olan devasa ölçekteki verileri dahi sorunsuz bir biçimde yönetebilecek yatay ölçeklenebilirlik (horizontal scalability) kapasitesine ulaşır.

Buckets (Kovalar) ve Sync Streams Mekanizması

Backend'deki milyonlarca satır verinin tüm kullanıcıların cep telefonlarına kopyalanması donanımsal ve mantıksal olarak imkansızdır. PowerSync, bu sorunu aşmak için veriyi kullanıcılara göre izole eden "Buckets" (Kovalar) mimarisini geliştirmiştir. Buckets, merkezi veritabanındaki verilerin belirli koşullara göre (örneğin "sadece user_id'si X olan kişinin profili ve o kişiye ait araçlar") filtrelenerek sadece ilgili istemcinin cihazına aktarılan alt veri kümeleridir. Sync Streams (Senkronizasyon Akışları) sayesinde bu bucket'lar kullanıcının cihazına sürekli olarak yansıtılır. Sistem bu eşitleme sırasında "Nedensel Tutarlılık" (Causal Consistency) prensibine göre çalışır; yani veriler arasındaki ilişkiyel bütünlük bozulmaz ve ana profil bilgisi cihaza inmeden o profile bağlı alt tabloların (örneğin araçların) öksüz bir şekilde cihaza inmesi engellenir.

İstemci (Client) Entegrasyonu: SupabaseConnector ve Veri Transferi

Supabase ve PowerSync mimarisinin Flutter veya diğer çapraz platform (cross-platform) dillerindeki istemci tarafında (frontend) çalışabilmesi için, uygulamanın çekirdek koduna PowerSyncBackendConnector soyut sınıfından türetilmiş bir köprü olan SupabaseConnector sınıfının implemente edilmesi gereklidir. Bu entegratör sınıf, istemci ile sunucu arasındaki tüm veri ve yetkilendirme trafiğini yöneten iki hayati asenkron fonksiyona ev sahipliği yapar.

fetchCredentials() Fonksiyonu ve Token Döngüsü

Cihaz ilk açıldığında, internet bağlantısı her koptuğunda veya cihaz uzun süre kapalı kaldıktan sonra uyandırıldığında PowerSync SDK'sının arka plan senkronizasyon servisine yetkili bir biçimde bağlanabilmesi için geçerli bir JSON Web Token (JWT) ve bağlantı uç noktasına (endpoint URL) ihtiyacı vardır. İşte bu noktada fetchCredentials() metodu devreye girer. Bu fonksiyon tetiklendiğinde, sistem doğrudan Supabase Auth modülüne başvurarak kullanıcının halihazırda geçerli olan güvenli oturum anahtarını (JWT) çeker ve PowerSync'e iletir. Eğer kullanıcı çevrimdışıyken (offline) bu token'ın süresi dolarsa, PowerSync SDK'sı akıllı bir şekilde beklemeye geçer. Cihaz tünele girip çıktığında veya Wi-Fi ağına tekrar bağlandığında fetchCredentials() otomatik olarak yeniden çağrılır, yeni ve taze bir token alınır ve

senkronizasyon hiç durmamış gibi kaldığı yerden devam eder. Bu mekanizma, Supabase'in kendi kullanıcı oturumu yaşam döngüsü ile PowerSync'in veri eşitleme döngüsünü birbirinden izole ederek hata toleransını maksimuma çıkarır.

uploadData() Fonksiyonu ve Veri Aktarımı

İkinci kritik operasyon, cihazdaki lokal değişikliklerin ana sunucuya gönderilmesini sağlayan uploadData() fonksiyonudur. Kullanıcı çevrimdışıyken örneğin araç plakasını sildiğinde bu eylem PowerSync'in lokal bir iş kuyruğuna (upload queue) alınır. İnternet erişimi sağlandığı anda bu fonksiyon devreye girer ve kuyrukta bekleyen tüm Oluşturma, Okuma, Güncelleme ve Silme (CRUD) işlemlerini bir paket (batch) halinde paketleyerek, Supabase'in sunucusuz API motoru olan PostgREST üzerinden buluta işler. Aktarım sırasında sunucu geçici olarak meşgulse veya bir hata oluşursa, sistem istisnaları yakalar ve veriyi kaybetmeden periyodik aralıklarla işlemi yeniden dener.

Çakışma Çözümleme (Conflict Resolution) Stratejileri

Bu tür dağıtık sistemlerde çözülmesi gereken en büyük mühendislik problemi, iki farklı cihazın çevrimdışıyken aynı anda aynı veriyi güncellemesi durumunda yaşanacak olan çakışmalardır. PowerSync, SupabaseConnector ile birlikte kullanıldığında varsayılan (default) olarak **LWW (Last-Write-Wins / Son Yazan Kazanır)** stratejisini kullanır. Yani iki farklı güncelleme geldiğinde, cihazlardan gönderilen zaman damgalarına (timestamp) bakılır ve en son kaydedilen değişiklik nihai doğru kabul edilerek diğerinin üzerine yazılır. Kimlik doğrulama, kullanıcı profili güncelleme veya araç bilgisi ekleme gibi "paylaşılan durum" (shared-state) senaryolarının tamamına yakını için bu LWW stratejisi mükemmel çalışır. Ancak daha kompleks ve eşzamanlı düzenleme gerektiren iş modellerinde, geliştiriciler LWW yerine CRDT'leri (Çakışmasız Çoğaltılmış Veri Türleri - Conflict-free Replicated Data Types) veya kendi özel birleştirme algoritmalarını (custom merge algorithms) sisteme kolayca entegre edebilirler.

Siber Güvenlik, JWT Altyapısı ve Yetkilendirme Kontrolleri

Oluşturulacak profillemeye ve kimlik doğrulama mimarisinin kalbini, kullanıcının yalnızca kendi verilerini görebilmesini garanti altına alan güvenlik ve yetkilendirme (authorization) protokolleri oluşturur. Veritabanında yasal düzenlemelere tabi olan araç şasi numaraları, plakalar ve mali işlemler için gerekli olan TCKN bilgileri saklanacağı için güvenlik duvarlarının yazılım kodu seviyesinde değil, doğrudan veritabanı motoru düzeyinde inşa edilmesi zorunludur.

Satır Bazlı Güvenlik (Row-Level Security - RLS) Mimarisi

Geleneksel yazılım mimarilerinde güvenlik kontrolleri genellikle backend kodunun içerisine yazılan koşullu ifadelerle (if-else blokları) sağlanır. Bu yaklaşım insan hatasına fazlasıyla açıktır ve veri sızıntılarının (data breaches) başlıca sebebidir. Supabase ve Postgres mimarisinde ise güvenlik doğrudan veritabanı tablolarına uygulanan **Satır Bazlı Güvenlik (Row-Level Security - RLS)** politikaları ile sağlanır.

RLS aktif edildiğinde, bir veritabanı tablosu her gelen sorguyu kaynağında doğrular. Örneğin vehicles tablosu üzerinde çalışacak bir RLS kuralı, sorguyu atan kullanıcının Supabase Auth

üzerinden ilettiği JWT token'ını çözer, içindeki sub (Subject / Kullanıcı ID) değerini okur ve sadece o ID ile eşleşen satırların okunmasına veya yazılmasına izin verir. Bu yapı, kötü niyetli bir siber saldırgan veya hatalı bir istemci kodu API üzerinden tüm araçları listelemek istese bile (SELECT * FROM vehicles), veritabanı motorunun yetkisi olmayan satırları otomatik olarak gizlemesiyle sonuçlanır. PowerSync sistemi, Supabase'in bu RLS altyapısı ile mükemmel bir uyum içinde çalışarak "Senkronizasyon Kuralları" (Sync Rules) mekanizmasını oluşturur. Bu sayede bir kullanıcının cihazına, kesinlikle ama kesinlikle başkasına ait hiçbir şasi numarası veya profil verisinin indirilmemesi matematiksel olarak garanti altına alınır.

JWT Kriptografik İmza Anahtarlarının (Signing Keys) Yönetimi

Supabase ve PowerSync arasındaki veri güvenliğini, kullanıcının kimliğini ispatlayan JSON Web Token (JWT) isimli kriptografik biletler sağlar. Bu token'ların başkaları tarafından üretilmesini veya içeriğinin değiştirilmesini engellemek için iki farklı anahtar (key) mimarisi mevcuttur ve sistem entegre edilirken doğru seçimin yapılması siber güvenlik standartları açısından kritik bir aşamadır:

Anahtar Mimarisi Tipi	Şifreleme Teknolojisi	Çalışma Mantığı ve Güvenlik Derecesi	PowerSync Entegrasyon Yöntemi
Yeni JWT Anahtarları (Asimetrik)	RSA / ECDSA (Asymmetric Cryptography)	Token'ı üreten Supabase, gizli anahtar (private key) ile imzalar. PowerSync gibi doğrulayıcılar, açık anahtar (public key / JWKS) kullanarak doğrular. Endüstri standardıdır ve en güvenli yöntemdir.	PowerSync paneline hiçbir gizli anahtar girilmez. JWKS (JSON Web Key Set) üzerinden sistem kendini otomatik yapılandırır.
Eski Tip Anahtarlar (Legacy Symmetric)	HS256 (Symmetric Cryptography)	Token'ı imzalayan da doğrulayan da aynı gizli şifreyi (shared secret) bilmek zorundadır. Gizli anahtarın çok fazla yerde bulunması güvenlik riski yaratır. Güvenlik düzeyi görece düşüktür.	Supabase'den kopyalanan JWT Secret anahtarı, PowerSync paneline manuel olarak "Legacy" alanına yapıştırılır.

Yeni kurgulanacak fazda, teknolojik borç (technical debt) yaratmamak adına kesinlikle Asimetrik (Yeni) JWT anahtarları kullanılmalıdır. Supabase üzerinde yerel geliştirme (local development) yaparken bile asimetrik anahtarların kullanımı için projenin config.toml dosyasında signing_keys_path konfigürasyonunun aktif edilmesi gerekmektedir. Bu altyapı tercihi, uygulamanın kimlik yönetim altyapısını kurumsal güvenlik (enterprise-grade security) seviyesine taşıyacaktır.

Sonuç ve Stratejik Mimari Tavsiyeler

Mevcut yazılım fazına entegre edilecek olan yeni nesil oturum açma ve kullanıcı profili yönetim

altyapısı; son kullanıcının sistemi hızlıca ve güvenle benimsemesini sağlayacak kadar sezgisel, çevrimdışı koşullarda kesintisiz veri işleyebilecek kadar dayanıklı (resilient) ve ağır regülasyonların talep ettiği tüm mahremiyet kurallarını karşılayacak kadar güvenli bir temele oturtulmalıdır. Bu süreçte hem teknik mekanizmalar hem de yasal sınırların gözetilmesi, projenin orta ve uzun vadedeki başarısının teminatıdır.

Yapılan detaylı sektörel, psikolojik, teknik ve hukuki analizler doğrultusunda sistem mimarisine ilişkin nihai stratejik öneriler şu şekilde sentezlenmektedir:

İlk olarak, kayıt mimarisinde köklü bir "Kademeli Kayıt" devrimi yapılmalıdır. Kullanıcı uygulamayla ilk tanıştığında, "Google ile Giriş" yöntemi bilişsel sürtünmeyi ortadan kaldırmak için tartışmasız birincil tercih olarak sunulmalı ve e-posta tabanlı kayıt, şifresiz doğrulama (Magic Link/OTP) desteğiyle alternatif olarak konumlandırılmalıdır. Kayıt ekranı sadece içeriye giriş kapısı olmalı; kullanıcının e-posta adresi dışında telefon numarası, TCKN veya araç şasi numarası (VIN) gibi yüksek değerli ve riskli veriler ilk fazda kesinlikle talep edilmemelidir. Profil oluşturma süreci "Kademeli Profilleme" (Progressive Profiling) disiplinine bağlanarak, her yeni veri ancak kullanıcı o verinin gerektirdiği bir işleme (örneğin fatura oluşturma veya aracı garaja ekleme) teşebbüs ettiğinde ve gerekçesi sunularak istenmelidir.

İkinci boyut olan Yasal Uyum ve Veri Minimizasyonu bağlamında, toplanan verilerin karakteri son derece ciddiye alınmalıdır. Araç şasi numarası ve araç plakası ilk bakışta nesnel veri gibi algılansa da, Avrupa Adalet Divanı'nın güncel kararları ve KVKK prensipleri doğrultusunda doğrudan bir kullanıcıya bağlandığı andan itibaren "kişisel veri" hüviyeti kazanmaktadır. TCKN ise, çok daha spesifik ve yüksek güvenlik gerektiren bir veri olup, meşru bir yasal zorunluluk bulunmayan hiçbir ekranda işlenmemeli; işlendiği noktalarda ise veritabanında durağan halindeyken (at rest) simetrik algoritmalarla şifrelenmelidir. Sistemde gereksiz veri barındırılmaması, olası bir veri ihlali anında idari cezaları ve prestij kayıplarını minimuma indirecek en etkili önlemdir.

Üçüncü teknik karar noktası ise sistemin senkronizasyon yetenekleridir. Mimari kesinlikle çevrimiçi öncelikli (online-only) dar boğazlardan arındırılmalı, Supabase ve PowerSync ikilisinin sunduğu "Çevrimdışı Öncelikli" (Offline-First) altyapıya geçilmelidir. Flutter veya ilgili istemci tarafında uygulanacak SupabaseConnector entegrasyonu ve SQLite üzerinden yerel veri yönetimi sayesinde, kullanıcıların internet erişiminin olmadığı yeraltı otoparklarında dahi plaka veya şasi güncellemesi yapabilmesi garanti altına alınmalıdır. Değişiklikler bağlantı sağlandığı an LWW (Son Yazan Kazanır) mantığıyla arka planda ana Postgres sunucusuna eşitlenmelidir. Son olarak, bu devasa veri akışı ve senkronizasyon ağının güvenliği, uygulamanın koduna terk edilmeyip veri kaynağında çözülmelidir. Supabase tarafında uygulanacak olan Satır Bazlı Güvenlik (RLS) politikaları sayesinde her bir satır verinin yetkilendirilmesi garanti edilmeli ve sistemdeki kimlik doğrulama biletlere (JWT), asimetrik kriptografi (RSA/ECDSA) standartlarına uygun biçimde yönetilmelidir. Özetle, ideal profilleme stratejisi "az bilgi, çok bağlam" yaklaşımına dayanmalı; teknik omurga ise çevrimdışı dahi hayatına devam edebilen, güvenlik ve mahremiyeti doğuştan (privacy-by-design) sistemin merkezine oturtan esnek bir mimariye dönüştürülmelidir.

Alıntılanan çalışmalar

1. What is considered personal data under the EU GDPR?, <https://gdpr.eu/eu-gdpr-personal-data/>
2. Can we identify an individual indirectly from the information we have (together with other available information)? | ICO, <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/personal-information-what-is-it/what-is-personal-data/can-we-identify-an-individual-indirectly/>
3. Offline-First Apps Made

Simple: Supabase + PowerSync,
<https://powersync.com/blog/offline-first-apps-made-simple-supabase-powersync> 4. PowerSync | Works With Supabase, <https://supabase.com/partners/integrations/powersync> 5. Supabase Auth - PowerSync Docs, <https://docs.powersync.com/configuration/auth/supabase-auth> 6. KVKK Kapsamında Veri Minimizasyonu İlkesi - Erdemir & Özmen, <https://www.erdemirozmen.com/hukukta-gundem/makaleler/kvkk-kapsaminda-veri-minimizasyonu-ilkesi> 7. Muddled Definitions of Personal Information Undermine Data Protection - FDD, <https://www.fdd.org/analysis/2023/12/26/muddled-definitions-of-personal-information-undermine-data-protection/> 8. Decoding Data? ECJ's verdict on Vehicle Identification Numbers as personal data, <https://www.twobirds.com/en/insights/2023/global/decoding-data-ecjs-verdict-on-vehicle-identification-numbers-as-personal-data> 9. CJEU considers whether Vehicle Identification Numbers constitute "Personal Data", <https://www.matheson.com/insights/cjeu-considers-whether-vehicle-identification-numbers-constitute-personal-data/> 10. TC KİMLİK NUMARASININ KİŞİSEL VERİ OLARAK İŞLENMESİ | Özgün Law Firm, <https://www.ozgunlaw.com/makaleler/tc-kimlik-numarasinin-kisisel-veri-olarak-islenmesi-4275> 11. Building Local-First Flutter Apps with Riverpod, Drift, and PowerSync | Dinko Marinac, <https://dinkomarinac.dev/blog/building-local-first-flutter-apps-with-riverpod-drift-and-powersync/> 12. PowerSync and Supabase: Just the Basics, <https://powersync.com/blog/powersync-and-supabase-just-the-basics> 13. Flutter Tutorial: Building An Offline-First Chat App With Supabase And PowerSync, <https://powersync.com/blog/flutter-tutorial-building-an-offline-first-chat-app-with-supabase-and-powersync> 14. Showcase: PowerSync For Supabase And Flutter, <https://powersync.com/blog/showcase-powersync-for-supabase-and-flutter> 15. PowerSync Setup Guide, <https://docs.powersync.com/intro/setup-guide> 16. Supabase - PowerSync Docs, <https://docs.powersync.com/integrations/supabase/guide> 17. Usage Examples - PowerSync Docs, <https://docs.powersync.com/client-sdks/usage-examples> 18. Kotlin SDK - PowerSync Docs, <https://docs.powersync.com/client-sdks/reference/kotlin> 19. powersync_client.dart - powersync_attachments_demo_app - GitHub, https://github.com/itsezlif/powersync_attachments_demo_app/blob/master/packages/powersync_client/lib/src/powersync_client.dart 20. Client-Side Integration With Your Backend - PowerSync Docs, <https://docs.powersync.com/configuration/app-backend/client-side-integration> 21. Authentication Setup - PowerSync Docs, <https://docs.powersync.com/configuration/auth/overview> 22. Integrating Supabase Authentication and PowerSync Chat into Existing Flutter App with GoRouter (application gets stuck on the loading screen) : r/flutterhelp - Reddit, https://www.reddit.com/r/flutterhelp/comments/1feem92/integrating_supabase_authentication_and_powersync/ 23. Get started quickly with PowerSync using Flutter and Supabase. - GitHub, <https://github.com/powersync-community/flutter-powersync-supabase>